

3. Conception de la solution

Cette partie présente les choix de conception effectués pour répondre aux besoins et contraintes formalisés en partie 2. Elle part de l'architecture générale du système pour descendre progressivement vers les décisions les plus spécifiques : la modélisation des données, l'abstraction des Training Resources, l'authentification, la communication avec l'application mobile, et enfin la gestion du fonctionnement hors ligne.

3.1. Architecture générale

La plateforme repose sur une architecture monolithique : un serveur unique Laravel héberge à la fois l'application web (destinée aux organisations et aux coaches) et l'API consommée par l'application mobile (destinée aux runners). Il n'y a pas de séparation physique entre ces deux façades, mais une séparation logique au niveau du routage et de l'authentification : un ensemble de routes sert l'interface web, un autre expose une API JSON dédiée au mobile, avec des mécanismes d'authentification distincts pour chacune.

Ce choix se justifie par le contexte du projet : une équipe restreinte (un développeur par partie), un produit en phase bêta commerciale, et un besoin de limiter la complexité technique. Le compromis assumé est un couplage plus fort entre web et mobile au sein d'une même base de code, en échange d'une simplicité de déploiement et de maintenance.

Les deux façades partagent la même couche métier (modèles, règles de permission, logique applicative), ce qui évite la duplication de logique entre elles — bénéfice concret du choix monolithique dans ce contexte.

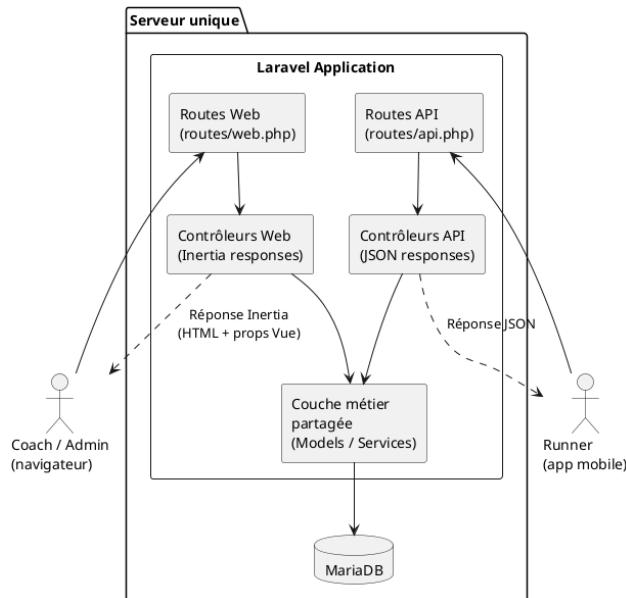


Figure 1: App Architecture

3.2. Choix technologiques et justification

Laravel / PHP et MariaDB n'ont pas fait l'objet d'une étude comparative formelle. Ils restent défendables au regard du besoin : le modèle de données du projet est fortement relationnel, sans exigence particulière qui aurait nécessité un autre SGBD ou un autre framework backend.

Inertia.js + Vue.js, en revanche, se justifie plus substantiellement à l'usage. Sans Inertia, il aurait fallu construire une véritable SPA consommant une API REST dédiée au web — donc maintenir deux API distinctes (web et mobile), avec deux logiques de sérialisation et deux couches d'autorisation à faire évoluer en parallèle. Avec Inertia, le serveur reste seul maître de l'état : chaque navigation renvoie directement un composant Vue avec ses données, sans exposer de JSON public réutilisable côté web. Seule l'API mobile reste alors une véritable API à concevoir et sécuriser comme telle.

C'est un compromis assumé : Inertia couple le frontend web aux routes Laravel, ce qui est acceptable puisque cette interface n'a pas vocation à être consommée autrement que par le navigateur — contrairement au mobile, qui nécessite une véritable API découplée.

L'authentification web (coach, organisation) repose sur le mécanisme de session natif de Laravel. Le runner, en revanche, requiert un système propre au projet : Sanctum, complété par un système de tokens custom ; ce choix est développé en 3.5, où il est directement lié aux contraintes du fonctionnement hors ligne du mobile.

3.3. Modélisation des données

Hierarchie des acteurs. Le modèle relationnel confirme et précise la hiérarchie des acteurs décrite en 2.5, chaque niveau étant modélisé comme une entité distincte plutôt que comme une variante d'un modèle utilisateur unique.

L'organisation dispose de ses propres identifiants de connexion, distincts de ceux de ses coaches — l'acteur « organisation » n'est donc pas un coach disposant d'un privilège supplémentaire, mais le compte du club lui-même. Elle porte également les quotas contractuels convenus à l'onboarding.

NoTrackRun elle-même est représentée comme une organisation, distinguée des organisations clientes par un indicateur dédié. C'est en son sein que sont rattachés les comptes des coaches admin. Le rôle admin est ainsi traité comme un cas particulier du modèle organisation/coach existant, plutôt que comme une exception à celui-ci.

Le runner, enfin, est le seul acteur sans identifiants de connexion propres à la plateforme web : il s'authentifie uniquement depuis l'application mobile, et ne porte aucune donnée d'identité au-delà d'un pseudonyme. Sa double relation à son coach et à son organisation permet — conformément à la règle métier de 2.5 — de le conserver au sein de l'organisation si son coach est supprimé, en vue d'une réaffectation ultérieure sans perte de son historique.

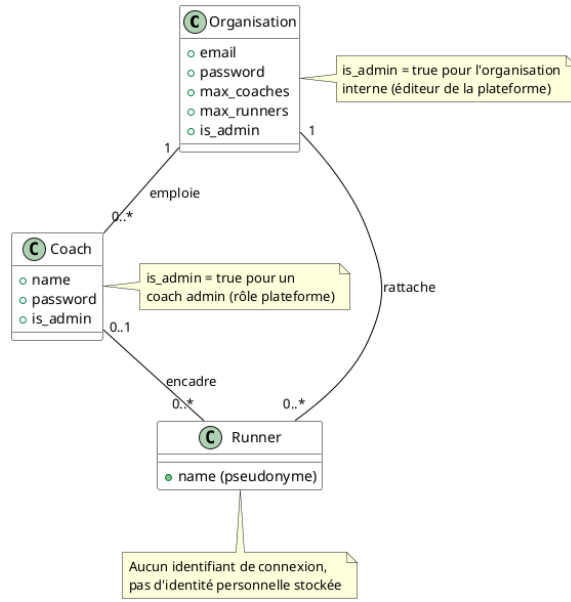


Figure 2: Diagramme des entités utilisateurs

Modélisation d'un plan d'entraînement : training plan vs user training plan. Le plan d'entraînement est modélisé en deux entités distinctes, qui répondent chacune à un besoin différent. Cette distinction est le choix de conception le plus structurant de tout le module de planification, et mérite d'être développée.

Le training plan est une ressource de coaching réutilisable, au même titre qu'un type de bloc ou un template de séance : il appartient à un coach, et ne fait référence à aucun runner en particulier. Ses séances ne sont pas positionnées à des dates fixes, mais à des coordonnées relatives — un numéro de semaine et un numéro de jour, formant un calendrier relatif qui peut être réutilisé pour n'importe quel runner, à n'importe quel moment. C'est cette structure relative qui permet à un même plan-modèle d'être assigné à plusieurs runners différents, à des dates de début différentes, sans avoir à en dupliquer la définition.

Le user training plan est créé au moment précis où un coach attribue un plan-modèle à un runner, avec une date de début choisie. À cet instant :

- le plan-modèle et l'ensemble de ses séances sont copiés dans une structure propre à ce runner, plutôt que simplement référencés ;
- la coordonnée relative (semaine/jour) de chaque séance copiée est conservée telle quelle, permettant de la replacer dans le calendrier réel du runner à partir de sa date de début propre.

Cette duplication, plutôt qu'une simple relation par référence, répond à trois besoins distincts :

1. **Indépendance vis-à-vis des modifications ultérieures du plan-modèle.** Si le coach fait évoluer le plan-modèle après l'avoir déjà assigné à un ou plusieurs runners, ces modifications ne doivent pas se répercuter rétroactivement sur ce que les runners ont déjà reçu — chacun garde la version qu'il avait au moment de l'assignation.
2. **Divergence indépendante par runner.** Deux runners assignés au même plan-modèle peuvent ensuite diverger l'un de l'autre : feedback, commentaires, suppressions ponctuelles d'une séance pour l'un sans affecter l'autre, alors qu'ils partagent la même origine.

Composition d'un plan d'entraînement. Qu'il s'agisse d'un plan-modèle ou d'un plan assigné, la structure interne d'un plan suit la même hiérarchie d'entités :

Entité	Rôle dans le plan
Plan	Regroupe l'ensemble des séances d'un programme d'entraînement, positionnées dans un calendrier relatif (semaine/jour).
Session	Une séance d'entraînement à réaliser un jour donné du plan ; regroupe les blocs qui la composent.
SessionType	Catégorise la séance (ex. Endurance, VMA, Récupération), indépendamment de son contenu détaillé.
Block	Unité de travail au sein d'une séance (ex. échauffement, intervalle, récupération) ; peut en contenir d'autres (besoin métier spécifique).
BlockType	Définit la nature d'un bloc (ex. Interval, Récupération) et détermine quels champs lui sont associés.
BlockField	Valeur planifiée par le coach pour un champ donné d'un bloc (ex. distance : 1000).
Unit	Unité de mesure dans laquelle une valeur de champ est exprimée (ex. mètres, minutes).
BlockResult	Résultat réellement exécuté par le runner pour un bloc (distance parcourue, durée, réussite), distinct de la valeur planifiée.

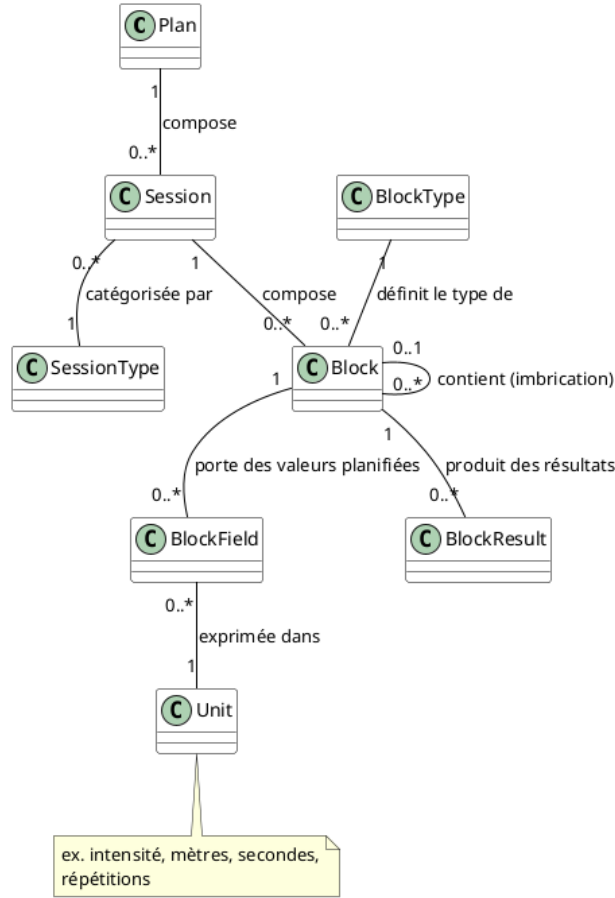


Figure 3: Diagramme des entités d'un plan d'entraînement

Une séance est composée d'un ou plusieurs blocs, ordonnés. Chaque bloc correspond à une occurrence concrète d'un type de bloc, c'est ce type qui détermine quels champs sont pertinents pour ce bloc : un bloc de type « Interval » attend par exemple une distance, une répétition et une intensité, alors qu'un bloc de type « Récupération » n'attend qu'une durée et une intensité. Chaque bloc porte ainsi les valeurs concrètes de ces champs pour cette occurrence précise : c'est la donnée planifiée par le coach, ce que le runner est censé réaliser.

Les blocs peuvent en outre être imbriqués : un bloc peut en contenir d'autres, ce qui répond à un besoin métier précis — permettre au coach d'insérer des accélérations ponctuelles à l'intérieur d'un bloc (par exemple une accélération de quelques secondes au sein d'un bloc d'endurance), sans avoir à modéliser cela comme une séquence de blocs indépendants au même niveau que le bloc qui les contient. Dans ce cas, c'est le bloc parent qui porte ses propres champs planifiés (par exemple la distance et l'intensité globales), tandis que ses sous-blocs représentent ces éléments ponctuels de façon indicative, sans être aujourd'hui individuellement suivis en résultat — seul le bloc parent fait l'objet d'un enregistrement de résultat. Cet usage de l'imbrication reste circonscrit à ce besoin pour l'instant, mais la structure sous-jacente ne l'y restreint pas : elle pourra être exploitée pour d'autres formes de composition à mesure que le besoin métier évolue.

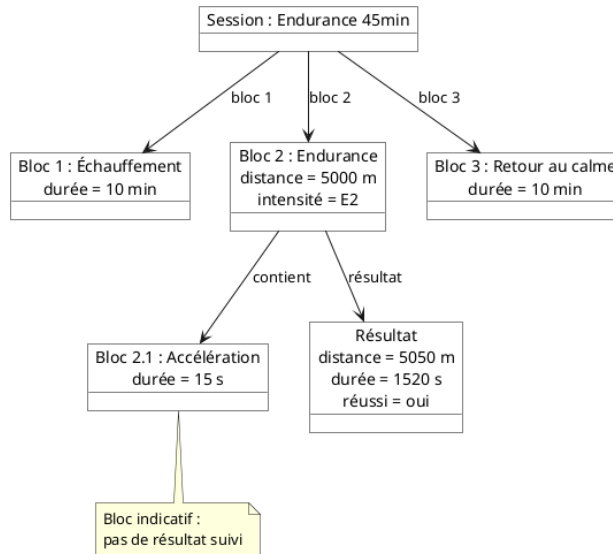


Figure 4: Structure d'une session

3.4. Conception des Training Resources

Le problème : des règles transversales à plusieurs types de ressources. Un type de block, un type de session, un template de session ou un plan sont des objets métier différents, mais qui posent tous exactement les mêmes questions : qui en est propriétaire, qui peut le consulter, le modifier, le supprimer, le copier, l'utiliser dans une séance ? Sans mutualisation, chacune de ces règles devrait être réécrite pour chaque type de ressource — avec le risque, à chaque évolution de la règle, de devoir la répercuter dans plusieurs classes en parallèle, au risque d'incohérences.

La plateforme répond à ce problème par une abstraction commune, `TrainingResource`, dont héritent l'ensemble des ressources de coaching réutilisables. Chaque type concret n'implémente que ce qui lui est propre ; la logique de propriété et de permission, elle, est écrite une seule fois.

Une abstraction organisée par comportement, pas par type. Plutôt qu'une seule interface générique, `TrainingResource` implémente plusieurs petites interfaces, chacune correspondant à un comportement précis : être visualisable, modifiable, supprimable, copiable, utilisable, et avoir un propriétaire.

Propriété et permissions calculées, non stockées. Aucune règle de permission n'est stockée comme attribut figé d'une ressource : tout est déduit au moment de la requête, à partir de deux éléments — le propriétaire de la ressource, et l'identité de l'utilisateur qui la consulte.

La propriété elle-même se décline à deux niveaux : une ressource appartient soit à un coach précis, soit directement à l'organisation (lorsqu'aucun coach n'y est rattaché). Un coach ne peut modifier ou supprimer qu'une ressource dont il est lui-même le créateur ; une organisation peut seulement réattribuer une ressource sans coach. La lecture, en revanche, est ouverte à l'ensemble des coaches d'une même organisation par défaut — ce qui confirme et rend concret le fonctionnement décrit en 2.5 : le partage au sein d'une organisation n'est pas une option à activer, c'est le comportement de base du système.

Concilier protection et flexibilité : la copie. La protection en modification pose un problème pratique : que faire si un coach souhaite partir d'une ressource protégée pour construire sa propre

variante ? C'est le rôle du comportement « copiable » : il permet à un coach de dupliquer n'importe quelle ressource qu'il peut consulter, la copie lui appartenant alors en propre et devenant pleinement modifiable. La protection ne verrouille donc jamais l'usage d'une ressource, seulement sa modification directe — un coach reste toujours libre d'en repartir pour construire la sienne.

Un second modèle de permission pour les ressources assignées. L'abstraction ne s'arrête pas aux ressources réutilisables du coach. Les entités assignées à un runner héritent également de `TrainingResource`, via une seconde classe intermédiaire qui redéfinit entièrement la logique de permission : l'accès n'y dépend plus de la propriété d'une ressource par un coach ou une organisation, mais de la relation d'encadrement entre le coach consultant et le runner concerné.

Ce choix illustre l'intérêt de l'abstraction par capacités plutôt que par héritage rigide : deux modèles de permission entièrement différents — l'un fondé sur la propriété et le partage, l'autre sur la relation coach/runner — cohabitent sous un même contrat, sans dupliquer la structure générale (calcul des droits, sérialisation vers le frontend) qui, elle, reste commune aux deux.

Une ressource protégée au service d'une logique algorithmique. Le statut de ressource protégée ne sert pas qu'à restreindre l'édition : il garantit aussi qu'une ressource exploitée par une logique algorithmique de la plateforme ne peut être altérée par un coach. C'est le cas de la séance-type utilisée pour le test de VMA, dont les résultats alimentent un calcul automatique — sa non-modifiabilité est une nécessité pour la fiabilité de ce calcul. Cette séance fait partie d'un socle de ressources de base fournies avec la plateforme dès l'onboarding d'une organisation, aux côtés d'autres ressources standards.

3.5. Authentification et autorisations

Un flow d'authentification différent selon l'acteur. Organisation et coach s'authentifient tous deux par un couple identifiant/mot de passe classique, le fonctionnement ne diffère pas d'une application web standard. Le runner, en revanche, n'a ni identifiant ni mot de passe : son parcours d'authentification est entièrement différent, et repose sur un système de tokens.

Le parcours du runner : de la création à l'usage courant. L'onboarding suit la hiérarchie déjà posée en 3.3 : l'organisation crée ses coaches, puis un coach crée un runner en lui attribuant simplement un pseudonyme — aucune information de contact ni mot de passe n'est demandée à ce stade.

1. **Activation.** À la création du runner, le système génère un token d'activation, à usage unique, que le coach transmet au runner. Celui-ci l'utilise une seule fois, au premier lancement de l'application mobile, pour rattacher l'appareil à son compte runner — c'est ce token qui remplace l'étape classique d'inscription par identifiant/mot de passe.
2. **Token d'accès.** L'activation réussie, l'application reçoit un token d'accès (durée de vie de 14 jours), conservé sur l'appareil. C'est ce token qui permet au runner de rester authentifié sur une longue période sans avoir à ressaisir quoi que ce soit — une nécessité directe du fonctionnement hors ligne prolongé décrit en 3.7.
3. **Token de session.** Pour chaque action effective sur la plateforme — récupérer un plan, envoyer des résultats — l'application présente son token d'accès afin d'obtenir un token de session, de durée de vie beaucoup plus courte (15 minutes) ; c'est ce dernier qui authentifie réellement les appels à l'API. Le token d'accès est lui-même à usage unique : chaque présentation déclenche, en plus du nouveau token de session, la génération d'un nouveau token d'accès qui remplace le précédent — une rotation glissante plutôt qu'un token fixe consommé

pendant toute sa durée de vie. La fenêtre de 14 jours se prolonge ainsi à chaque usage effectif de l'application, plutôt que de courir immuablement depuis l'activation initiale.

Que se passe-t-il à l'expiration du token d'accès ? Passé les 14 jours, le token d'accès expire sans renouvellement automatique : le runner ne peut plus obtenir de nouveau token de session, et donc plus interagir avec la plateforme. Il doit alors s'adresser à son coach pour qu'un nouveau token d'activation lui soit généré, reproduisant la première étape du parcours. Côté web, le coach est alerté lorsqu'un token de l'un de ses runners arrive à expiration.

diagramme de séquence parcours d'authentification du runner en annexe.

3.6. Conception de l'API et communication avec le mobile

Cette section porte sur les principes de conception de l'API mobile — la nature des échanges et la forme des données.

Endpoint	Rôle
POST /users/activate	Active l'application avec le token d'activation
GET /user/training-plans	Récupère l'ensemble des plans assignés au runner
POST /user/session-result/{session}	Envoie les résultats d'une séance exécutée
POST /user/session-review/{session}	Envoie le feedback qualitatif du runner sur une séance
POST /user/bug-report	Permet de signaler un problème directement depuis l'application

Le dernier endpoint mérite une remarque : il illustre concrètement la nature de produit commercial en production du projet, plutôt qu'un exercice académique — un canal de retour existe pour que les runners signalent un dysfonctionnement directement depuis l'application.

Un seul appel, un payload consolidé. GET /user/training-plans ne renvoie pas un plan à la fois, mais l'ensemble des plans actuellement assignés au runner, chacun avec l'intégralité de sa hiérarchie déjà résolue — séances, blocs, champs planifiés et résultats déjà enregistrés. Le mobile n'a donc aucune reconstruction à faire à partir de fragments : un seul appel suffit à obtenir tout ce dont l'application a besoin pour fonctionner hors ligne pendant une période prolongée.

Un détail de conception important : les séances sont transmises avec une date absolue déjà calculée ("date": "2026-03-10"), et non le couple semaine/jour relatif utilisé en base. C'est le serveur qui résout ce calcul à partir de la date de début du plan assigné, avant l'envoi — le mobile n'a ainsi aucune logique de calendrier à reproduire, ce qui réduit d'autant la surface de désynchronisation possible entre les deux implémentations.

Payload réel complet en annexe.

Transmission des résultats. À l'inverse, l'envoi des résultats se fait séance par séance plutôt que plan entier, avec un payload volontairement minimal — un point déjà présenté en amont du

document (index, distance, durée, réussite, identifiant du bloc concerné), qui correspond directement à la structure `BlockResult`.

Format de réponse et validation. Toutes les réponses de l'API suivent une enveloppe commune (success, error, data, status), qu'il s'agisse d'un succès ou d'une erreur métier — un principe de conception simple qui uniformise le traitement des réponses côté mobile, indépendamment de l'endpoint appelé. Les règles de validation des données entrantes (formats attendus, contraintes métier) sont, elles, définies au niveau de la conception mais mises en œuvre techniquement en 4.4.

3.7. Fonctionnement hors ligne et synchronisation

Le problème. Le runner récupère (pull) son plan puis peut rester hors ligne pendant une période prolongée — plusieurs semaines dans certains cas — avant de renvoyer (push) ses résultats. Pendant cette période, rien n'empêche le coach de continuer à faire évoluer ce même plan côté serveur : modifier, voire supprimer, une séance ou un bloc que le runner a déjà en local et est en train — ou a déjà fini — d'exécuter.

Le système doit donc composer avec une divergence possible entre l'état du plan tel que le serveur le connaît au moment du push, et l'état tel que le runner l'a réellement exécuté, sans pour autant perdre les résultats transmis ni compromettre l'intégrité de la base.

diagramme de séquence illustrant push / pull et divergence en annexe.

La solution actuelle : suppression logique et restauration. Pour garantir que les résultats du runner puissent toujours être associés à un bloc et à une séance valides en base, le système repose sur une suppression logique (soft delete) plutôt que physique : une séance ou un bloc « supprimé » par le coach n'est pas réellement effacé, et se trouve restauré automatiquement dès que des résultats le concernant arrivent — l'intégrité référentielle du côté du push reste ainsi toujours garantie, quel que soit l'état dans lequel le coach a laissé le plan entre-temps.

Limite actuelle : une traçabilité insuffisante. Le plan assigné porte deux horodatages — dernier pull, dernier push — qui indiquent quand le runner a synchronisé, mais pas quelle version exacte du plan il avait alors en main. Si le coach modifie un plan après qu'un runner l'a récupéré, rien aujourd'hui ne permet de reconstituer précisément ce que ce dernier a effectivement reçu et exécuté, au-delà de ce qu'il a bien voulu renvoyer comme résultats.

Comblers cette limite suppose de conserver, d'une manière ou d'une autre, l'état du plan tel qu'il était à un instant donné — une préoccupation différente de la suppression logique déjà en place (qui ne couvre que les suppressions, pas les modifications de valeur). C'est ce besoin qui a motivé une première tentative, testée puis abandonnée, avant qu'une piste alternative ne soit retenue.

Tentative abandonnée : un event sourcing hybride. Une première approche par event sourcing a été explorée : des événements étaient déclenchés manuellement dans les contrôleurs à chaque modification d'un bloc. Chaque événement portait un payload contenant l'état complet et dénormalisé du bloc concerné à cet instant — libellé, couleur, champs, unités, icônes — associé à un utilisateur, une entité et un type d'événement.

Ce n'était pas un event sourcing « pur » : le payload de chaque événement contenait déjà un état complet d'une seule entité, et non un delta compact. C'était un hybride, empruntant la granularité fine d'un log d'événements sans le bénéfice structurel d'un vrai event sourcing (deltas compacts, replay incrémental).

Deux problèmes concrets ont motivé l’abandon de cette piste :

1. **Un décalage avec le format attendu côté mobile.** Le mobile attend un payload consolidé — le plan entier, en un seul lot (cf. 3.6) — pas une série de micro-événements à rejouer. Reconstruire « ce que le runner a reçu » aurait nécessité de retrouver tous les événements pertinents antérieurs à la date de pull, de ne garder que le dernier par bloc, puis de réassembler manuellement toute la hiérarchie plan → séance → bloc. Un travail de reconstruction en lecture plus lourd que ce que le pattern est censé apporter, sans bénéficier de son intérêt réel.
2. **Un déclenchement dispersé dans les contrôleurs.** Les événements étaient créés manuellement à chaque point d’entrée touchant un bloc — ce qui suppose de penser à les déclencher partout où c’est pertinent (mise à jour directe, suppression en cascade, actions admin...), avec un risque de capture incomplète si un point d’entrée est oublié. C’est exactement le même risque que celui identifié plus haut pour la restauration actuelle, aujourd’hui déclenchée dans un unique contrôleur : la fiabilité d’un mécanisme de traçabilité dépend d’une capture garantie et centralisée, pas dispersée au gré des contrôleurs.

L’évent sourcing, même rendu plus « pur », aurait aggravé le problème plutôt que de le résoudre : le besoin côté mobile est grossier (un état consolidé par pull), alors que le pattern vise une granularité fine adaptée à l’audit détaillé, pas à la reconstruction rapide d’un gros payload cohérent.

Piste retenue : reconstruction par date (SCD Type 2). Deux options restent ouvertes pour combler la limite de traçabilité, sans reproduire l’erreur de la tentative précédente.

Le snapshot matérialisé consisterait à dupliquer, à chaque pull, le payload consolidé exactement tel qu’envoyé au mobile — pas de reconstruction ni de replay, juste une copie figée. C’est l’option la plus simple et la plus sûre à mettre en œuvre.

La reconstruction par date, retenue comme piste de conception à approfondir, consiste à ne rien dupliquer et à reconstituer l’état du plan à la demande, en filtrant les données actuelles à partir de la dernière date de pull du runner. Il s’agit d’un principe de SCD (Slowly Changing Dimension) Type 2 — à distinguer explicitement de l’évent sourcing testé plus haut : on y stocke des versions d’état bornées dans le temps, et connaître l’état à une date T se fait par une simple requête filtrée, sans calcul ni rejeu. C’est précisément ce que la tentative event sourcing n’était pas, malgré une structure de stockage en apparence proche (un log horodaté) : chaque événement y portait déjà un état complet plutôt qu’un delta, ce qui imposait quand même un filtrage puis une déduplication en lecture — une reconstruction plus lourde qu’un simple filtre SCD, sans les bénéfices structurels de l’un ou l’autre pattern.

Pour les suppressions, le soft delete déjà en place (décrit ci-dessus) constitue une brique directement réutilisable pour ce filtre par date. Pour les modifications, en revanche, rien n’existe encore : une mise à jour classique écrase la valeur précédente, qui devient irrécupérable pour une reconstruction a posteriori — il faudrait faire évoluer les écritures vers un versionnement, au minimum sur les entités concernées par la reconstruction.

Point de vigilance. Le risque de capture dispersée déjà identifié pour la restauration actuelle et pour la tentative event sourcing s’appliquerait de la même façon à un versionnement SCD des modifications s’il était, lui aussi, déclenché au coup par coup dans chaque contrôleur. Centraliser la logique d’écriture (service dédié, ou observer Eloquent sur le modèle) plutôt que de la disperser est une condition de fiabilité du mécanisme, pas un détail secondaire — un point à traiter dès la conception plutôt qu’à corriger après coup.

Décision retenue à court terme : snapshot borné, avec purge des périodes inactives. La

décision effectivement retenue à court terme combine les deux options plutôt que de trancher entre elles. Le principe reste celui du snapshot matérialisé — le payload consolidé est dupliqué tel quel à chaque pull, sans reconstruction ni rejeu — mais chaque snapshot est désormais borné dans le temps par une période de validité, à la manière d'un SCD Type 2 : au moment où un nouveau pull donne lieu à un nouveau snapshot (si le plan a changé depuis le précédent), la période de validité du snapshot précédent est close à cette date, et celle du nouveau démarre.

Une règle de purge vient s'ajouter à ce principe : lorsqu'un nouveau snapshot est créé, si aucun résultat n'a été transmis par le runner pendant toute la période de validité du snapshot précédent, ce dernier est supprimé plutôt que conservé. L'idée est simple : un snapshot qui n'a jamais servi de référence pour un résultat réellement exécuté n'apporte aucune valeur d'audit — il ne fait que représenter une période où le runner a peut-être ouvert l'application sans rien exécuter (un pull répété sans usage réel, par exemple). Ne conserver que les snapshots qui ont effectivement « couvert » une activité du runner permet de garder une trace utile pour le coach, sans laisser le stockage croître au rythme de chaque pull, y compris ceux qui ne correspondent à aucun usage concret.